UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Memory Management

Mirco Marchetti, Riccardo Lancellotti

SECloud research group

University of Modena and Reggio Emilia

AY 2025/26

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

Background

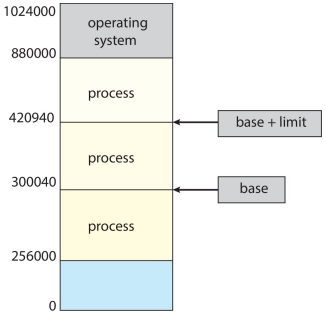
Background

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Program must be brought (from disk) into memory and placed within a process for it to be run
- CPU can access directly only Main memory and registers
- Memory unit sees a stream of
  - Addresses and read requests
  - Address + data and write requests
- Register access is done $\leq$ CPU clock
- Main memory can take many cycles, causing a stall
- Cache sits between main memory and CPU registers
- Protection of memory required to ensure correct operation

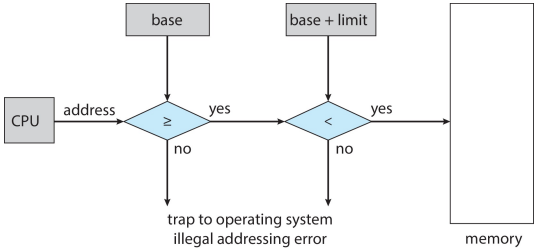
Protection

- Need to ensure that a process can access only those addresses in its address space.
- We can provide this protection by using a pair of base and limit registers define the logical address space of a process



Hardware Address Protection

- CPU must check every memory access generated in user mode to be sure it is between base and limit for that user
- Instructions to load base and limit registers are privileged



Address Binding

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

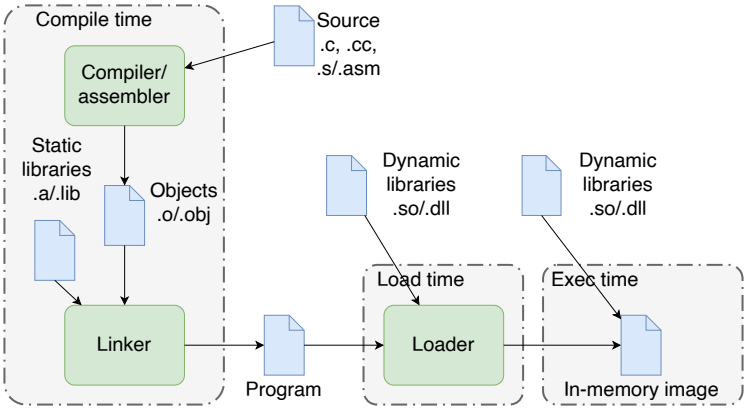
- Programs on disk, ready to be brought into memory to execute form an input queue
  - Without support, must be loaded into address 0000
- Inconvenient to have first user process physical address always at 0000
  - Alternative approaches have been designed
- Addresses represented in different ways at different stages of a program's life
  - Source code addresses usually symbolic
  - Compiled code addresses bind to relocatable addresses
    i.e.: 14 bytes from beginning of this module
  - Linker or loader will bind relocatable addresses to absolute addresses
    i.e.: 74014
  - Each binding maps one address space to another

Binding to Memory

Address binding of instructions and data to memory addresses can happen at three different stages

- Compile time: If memory location known a priori, absolute code can be generated; must recompile code if starting location changes
- Load time: Must generate relocatable code if memory location is not known at compile time
- Execution time: Binding delayed until run time if the process can be moved during its execution from one memory segment to another
  - Need hardware support for address maps (e.g., base and limit registers)

Multistep Processing of a User Program



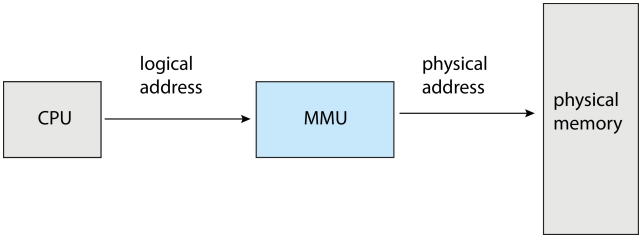
Logical vs. Physical Address Space

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- The concept of a logical address space that is bound to a separate physical address space is central to proper memory management
  - Logical address: generated by the CPU; also referred to as virtual address
  - Physical address: address seen by the memory unit
- Logical and physical addresses are the same in compile-time and load-time address-binding schemes; logical (virtual) and physical addresses differ in execution-time address-binding scheme
- Logical address space is the set of all logical addresses generated by a program
- Physical address space is the set of all physical addresses generated by a program

Memory-Management Unit (MMU)

- Hardware device that at run time maps virtual to physical address
- Many methods possible, covered in the rest of this chapter

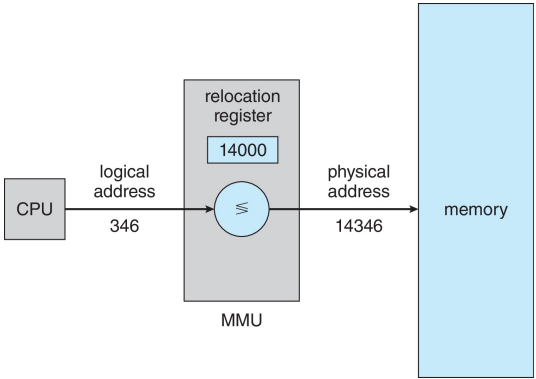


Memory-Management Unit

- Consider generalization of the base-register scheme
- Base register called relocation register in this context
- Value in the relocation register is added to every address generated by a user process at the time it is sent to memory
- User program deals with logical addresses; it never sees the real physical addresses
  - Execution-time binding occurs when reference is made to location in memory
  - Logical address bound to physical addresses

Memory-Management Unit

- Consider generalization of the base-register scheme
- Base register now called relocation register
- The value in the relocation register is added to every address generated by a user process at the time it is sent to memory



Dynamic Loading

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- The entire program does need to be in memory to execute
- Routine is not loaded until it is called
- Better memory-space utilization; unused routine is never loaded
- All routines kept on disk in relocatable load format
- Useful when large amounts of code are needed to handle infrequently occurring cases
- No special support from the operating system is required
  - Implemented through program design
  - OS can help by providing libraries to implement dynamic loading

Linking

- Static linking: system libraries and program code combined by the loader into the binary program image
  - On Unix static libraries are archives containing multiple object files (`.a` extension)
  - On Windows have extension `.lib`
- Dynamic linking: linking postponed until execution time

Dynamic Linking

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Small piece of code, stub, used to locate the appropriate memory-resident library routine in program memory
  - On Unix the stub is generated at compile time from the dynamic library
- Dynamic linking libraries (`.dll`) also known as shared objects (`.so`)
  - Versioning may be needed
- At run-time a linking occurs
  - Stub is replaced with the address of the routine
  - Run-time linking is executed by a special library named `ld.so`
  - Known libraries location is stored in a file named `/etc/ld.so.cache`
  - The system looks for known libraries in paths stored in `/etc/ld.so.conf`
- Additional libraries can be requested later during execution (e.g., plugins)
  - `dlopen` system call – not discussed here

Shared Objects and Symlinks

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Shared libraries stored in standard directories (e.g., `/lib`, `/usr/lib`)
- Symlinks (symbolic links) used to manage multiple versions
  - Actual library: `libjpeg.so.62.3.0`
  - Symlink: `libjpeg.so.62` → `libjpeg.so.62.3.0`
  - Symlink: `libjpeg.so` → `libjpeg.so.62`
- Linker searches for libraries using symlinks
- Allows version updates without recompiling programs
- Program linked against `libjpeg.so` automatically uses current version

Symlink Resolution Process

- Dynamic linker examines `DT_NEEDED` entries in executable
- For each library name, searches standard paths
- Resolves symlinks to actual library file
- Loads resolved library into virtual address space
- Multiple programs can map same library via different symlink paths
- `ldconfig` utility rebuilds symlink cache for faster resolution
  - Creates `/etc/ld.so.cache` with symlink mappings

Identifying Library Dependencies with `ldd`

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- `ldd` command lists dynamic dependencies of an executable
- Shows which shared libraries are needed and their paths
- Example output:

```
$ ldd /usr/bin/gcc
linux-vdso.so.1 (0x00007f6c1c398000)
libc.so.6 => /usr/lib/x86_64-linux-gnu/libc.so.6 (0x00007f6c1c167000)
/lib64/ld-linux-x86-64.so.2 (0x00007f6c1c39a000)
```

- Each line shows: library name, resolved symlink path, memory address
- Helps verify all dependencies are available
- Can identify missing libraries (marked as `not found`)
- Can look for dynamic symbols also using the `nm -D` command

Examining Symbols in Shared Objects

- `nm` command lists symbols in object files and libraries
- Shows exported functions and variables
- Example: examining symbols in `libc.so.6`

```
nm -D /lib/x86_64-linux-gnu/libc.so.6 | head -n 5
0000000000041d60 T a64l@@GLIBC_2.2.5
000000000002848e T abort@@GLIBC_2.2.5
00000000001eab40 B __abort_msg@@GLIBC_PRIVATE
0000000000041e60 T abs@@GLIBC_2.2.5
00000000001165d0 T accept@@GLIBC_2.2.5
```

- `-D` flag shows dynamic/exported symbols only
- Column meaning: offset, symbol type, symbol name

Symbol Types in `nm` Output

Type	Description
`T/t`	Text section (code/executable)
`D/d`	Data section (initialized global data)
`B/b`	BSS section (uninitialized data)
`R/r`	Read-only data section
`U`	Undefined symbol (external reference)
`V/v`	Weak object (can be left undefined)
`W/w`	Weak symbol (can be left undefined)
`a`	Absolute symbol (fixed address)

- Uppercase letters indicate global symbols; lowercase indicate local symbols
- `U` symbols must be resolved by linker from other objects or libraries
  - Suggestion: look with `nm -D` at the `hello_world` example

Comparing `.a` and `.so` Symbols

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Static archives (`.a`) containing object files
  - The option `-A` for `nm` prints the object file within the library
  - can list the content of the archive with `at t <library_file.a>`
- Shared objects (`.so`) are dynamic linking libraries
- Compare symbols using `nm`:

```
nm -A libjpeg.a| grep jpeg_read_header
libjpeg.a:jdapimin.c.o:0000000000000390 T jpeg_read_header

nm -D libjpeg.so| grep jpeg_read_header
000000000001e4b0 T jpeg_read_header@@LIBJPEG_6.2
```

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

Contiguous Allocation

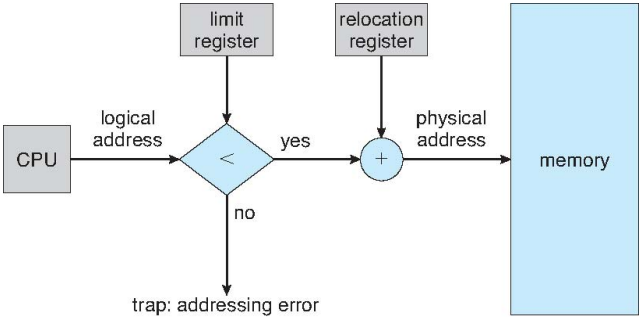
Contiguous Allocation

- Main memory must support both OS and user processes
- Limited resource, must allocate efficiently
- Contiguous allocation is one early method
- Main memory usually into two partitions
  - Resident operating system, usually held in low memory with interrupt vector
  - User processes then held in high memory
  - Each process contained in single contiguous section of memory

Contiguous Allocation

- Relocation registers used to protect user processes from each other, and from changing operating-system code and data
- Base register contains value of smallest physical address
- Limit register contains range of logical addresses
  - Each logical address must be less than the limit register
- MMU maps logical address dynamically
- Can then allow actions such as kernel code being transient and kernel changing size

Simple MMU


limit register
relocation register
logical address
CPU
<
yes
+
physical address
memory
no
trap: addressing error

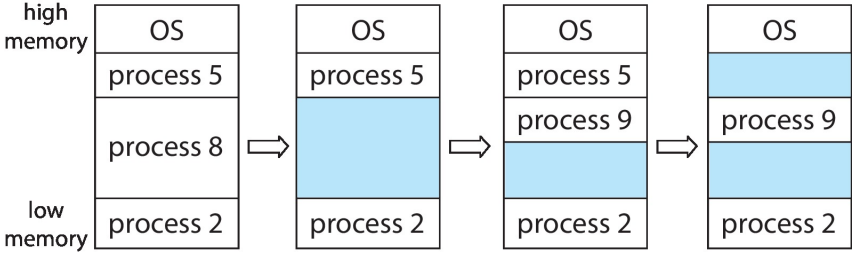
Variable Partition

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

Multiple-partition allocation

- Degree of multiprogramming limited by number of partitions
- Variable-partition sizes for efficiency (sized to a given process' needs)
- Hole – block of available memory; holes of various size are scattered throughout memory
- When a process arrives, it is allocated memory from a hole large enough to accommodate it
- Process exiting frees its partition, adjacent free partitions combined
- Operating system maintains information about:
  - Allocated partitions
  - Free partitions (hole)

Variable Partition



Dynamic Storage Allocation

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

How to satisfy a request of size *n* from a list of free holes?

- First-fit: Allocate the first hole that is big enough
- Best-fit: Allocate the smallest hole that is big enough; must search entire list, unless ordered by size
  - Produces the smallest leftover hole
- Worst-fit: Allocate the largest hole; must also search entire list
  - Produces the largest leftover hole
- First-fit and best-fit better than worst-fit in terms of speed and storage utilization

Fragmentation

- External Fragmentation: total memory space exists to satisfy a request, but it is not contiguous
- Internal Fragmentation: allocated memory may be slightly larger than requested memory;
  - Size difference: memory internal to a partition, but not being used
- First fit analysis reveals that given N blocks allocated, $0.5N$ blocks lost to fragmentation
  - 1/3 may be unusable $\rightarrow$ 50% rule

Fragmentation

- Reduce external fragmentation by compaction
- Shuffle memory contents to place all free memory together in one large block
- Compaction is possible only if relocation is dynamic, and is done at execution time
- I/O problem
  - Latch job in memory while it is involved in I/O
  - Do I/O only into OS buffers
- Note: backing store has same fragmentation problems

Use of contiguous allocation

- Mostly replaced by paging strategies
- Still critical in some cases
- Direct Memory Access (DMA)
  - Legacy devices do not support paging in memory access
  - In Linux Contiguous Memory Allocator (CMA) is used to reserve large blocks of physical memory for these devices
- Embedded Systems and Real-Time OS
  - Contiguous allocation is computationally cheap. The OS only needs to track a starting address and a limit
  - In Real-Time systems, you need to know exactly how long a memory access will take. Paging can introduce unpredictable delays
- Video/Multimedia Processing
  - Support for very large buffers: a single uncompressed 4K frame can exceed 24MB
  - Hardware video encoders and decoders require these buffers to be contiguous to process the data at the high speeds

Use of contiguous allocation

- Contiguous allocation is not widely used at OS-level
- Principles of contiguous allocation are still valid for user-space memory management
- What happens when we invoke `malloc()` and `free()`
  - Can operate on memory break (upper limit of program memory limit)
  - Can use the `brk()` system call to increase process memory space
  - Can rely on previously allocated-and-released memory approaches
  - Conceptually identical to the problems of contiguous allocation in OS
  - Will be seen in more advanced courses

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

Paging

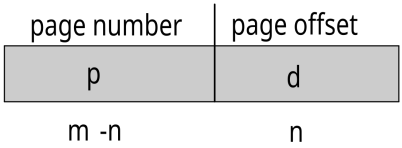
Paging

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

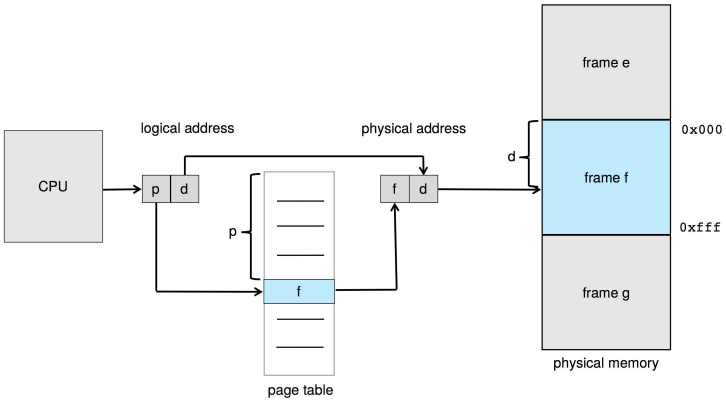
- Physical address space of a process can be noncontiguous; process is allocated physical memory whenever the latter is available
  - Avoids external fragmentation
  - Avoids problem of varying sized memory chunks
- Divide physical memory into fixed-sized blocks called frames
  - Size is power of 2, between 512 bytes and 16 Mbytes
- Divide logical memory into blocks of same size called pages
- Keep track of all free frames
- To run a program of size *N* pages, need to find *N* free frames and load program
- Set up a page table to translate logical to physical addresses
- Backing store likewise split into pages
- Still have Internal fragmentation

Address Translation Scheme

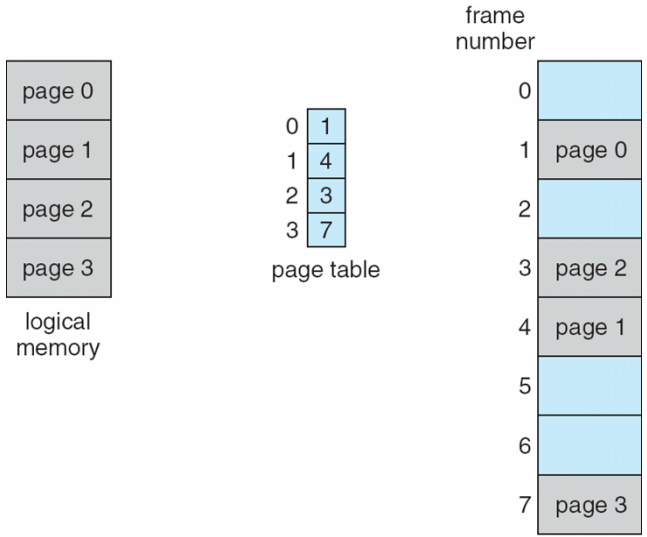
- Address generated by CPU is divided into
- Page number *p*: used as an index into a page table which contains base address of each page in physical memory
- Page offset *d*: combined with base address to define the physical memory address that is sent to the memory unit
- For given logical address space 2^m and page size 2^n



Paging Hardware

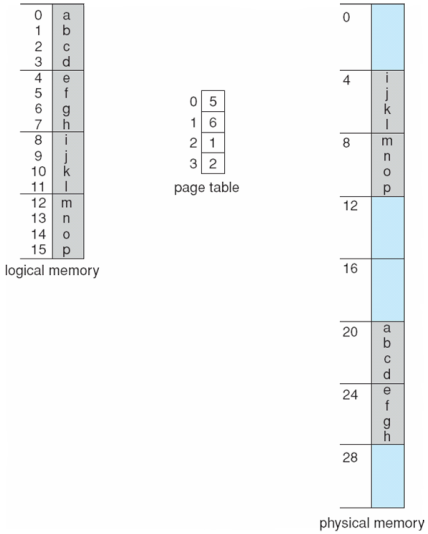


Paging Model



Paging Example

- Logical address: n = 2 and m = 4.
- Page size of 4 bytes and
- Physical memory of 32 bytes (8 pages)



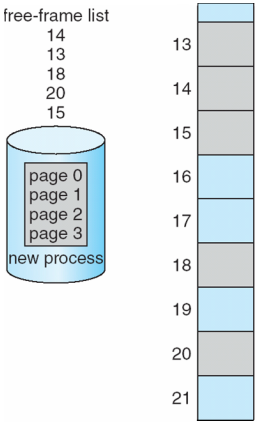
Calculating internal fragmentation

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

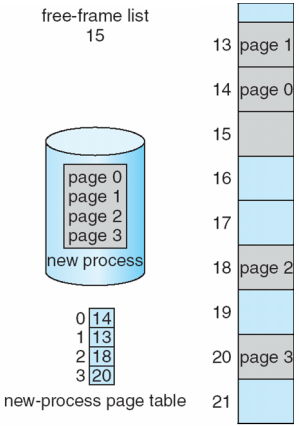
- Page size = 2,048 bytes
- Process size = 72,766 bytes
- 35 pages + 1,086 bytes
- Internal fragmentation of 2,048 - 1,086 = 962 bytes
- Worst case fragmentation = 1 frame – 1 byte
- On average fragmentation = 1 / 2 frame size
- So small frame sizes desirable?
- But each page table entry takes memory to track
- Page sizes growing over time
  - Solaris supports two page sizes – 8 KB and 4 MB
  - Linux supports several page sizes – depend on underlying harware

Free Frames

Before allocation



After allocation



Implementation of Page Table

- Page table is kept in main memory
  - Page-table base register (PTBR) points to the page table
  - Page-table length register (PTLR) indicates size of the page table
- In this scheme every data/instruction access requires two memory accesses
  - One for the page table and one for the data / instruction
- The two-memory access problem can be solved by the use of a special fast-lookup hardware cache called translation look-aside buffers (TLBs) (also called associative memory).

Translation Look-Aside Buffer

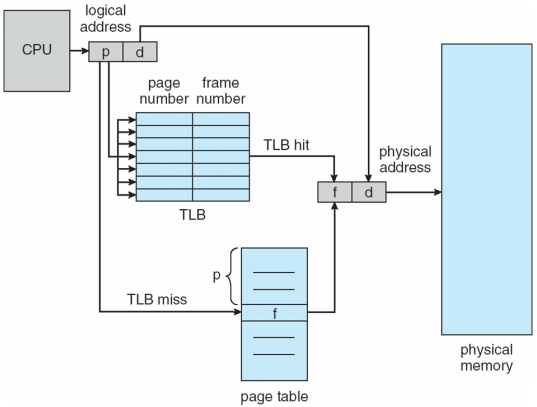
- Some TLBs store address-space identifiers (ASIDs) in each TLB entry – uniquely identifies each process to provide address-space protection for that process
  - Otherwise need to flush at every context switch
- TLBs typically small (64 to 1,024 entries)
- On a TLB miss, value is loaded into the TLB for faster access next time
  - Replacement policies must be considered
  - Some entries can be wired down for permanent fast access

Hardware

- Associative memory – parallel search
- Address translation (*p*, *d*)
  - If *p* is in associative register, get frame # out
  - Otherwise get frame # from page table in memory

Page #	Frame #

Paging Hardware With TLB



Effective Access Time

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Hit ratio – percentage of times that a page number is found in the TLB
  - An 80% hit ratio means that we find the desired page number in the TLB 80% of the time.
- Suppose that 10 nanoseconds to access memory.
  - If we find the desired page in TLB then a mapped-memory access take 10 ns
  - Otherwise we need two memory access so it is 20 ns
- Effective Access Time (EAT)
  - EAT = 0.80 x 10 + 0.20 x 20 = 12 nanoseconds
  - 20% slowdown in access time
- Consider a more realistic hit ratio of 99%,
  - EAT = 0.99 x 10 + 0.01 x 20 = 10.1ns
  - implying only 1% slowdown in access time

Retrieving Page Size on Linux

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Page size is a system configuration parameter
- Can be retrieved using several methods
  - `getpagesize()` or `sysconf()` functions
  - `getconf` command
- Program to retrieve page size

```c
#include <unistd.h>
#include <stdio.h>
int main() {
  long page_size = getpagesize();
  printf("Page size: %ld bytes\n", page_size);
  return 0;
}
```

- Compile and run:

```
make pagesize && ./pagesize
Page size: 4096 bytes
```

Retrieving Page Size: Alternative Methods

- Using `sysconf()` for portable code

```
#include <unistd.h>
long page\_size = sysconf(\_SC\_PAGESIZE);
```

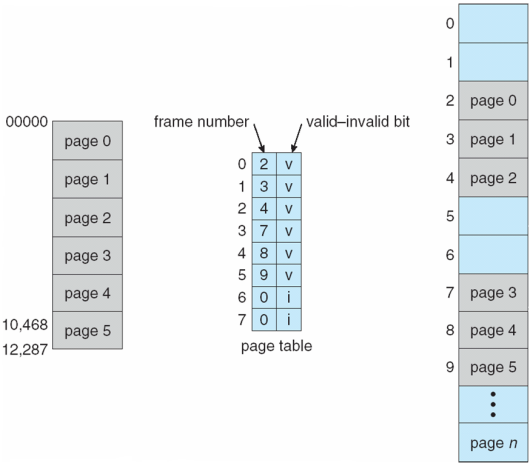
- Using shell command `getconf`

```
$ getconf PAGESIZE
4096
$ getconf PAGE_SIZE
4096
```

Memory Protection

- Memory protection implemented by associating protection bit with each frame to indicate if read-only or read-write access is allowed
- Can also add more bits to indicate page execute-only, and so on
- Valid-invalid bit attached to each entry in the page table
  - Valid indicates that the associated page is in the process' logical address space, and is thus a legal page
  - Invalid indicates that the page is not in the process' logical address space
  - Or use page-table length register (PTLR)
- Any violations result in a trap to the kernel

Valid (v) or Invalid (i) Bit In A Page Table

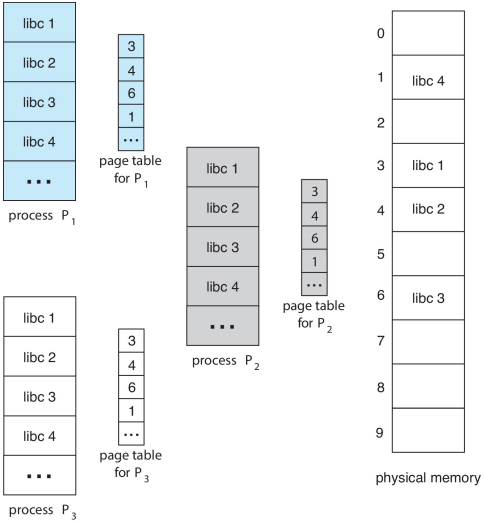


Shared Pages

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Shared code
  - One copy of read-only (reentrant) code shared among processes (i.e., text editors, compilers, window systems)
  - Similar to multiple threads sharing the same process space
  - Also useful for interprocess communication if sharing of read-write pages is allowed
- Private code and data
  - Each process keeps a separate copy of the code and data
  - The pages for the private code and data can appear anywhere in the logical address space

Shared Pages Example



Structure of the Page Table

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

Memory structures for paging can get huge using straight-forward methods

- Consider a 32-bit logical address space as on modern computers
- Page size of 4 KB (2^{12})
- Page table would have 1 million entries (2^{32} / 2^{12})
- If each entry is 4 bytes → each process 4 MB of physical address space for the page table alone
  - Don't want to allocate that contiguously in main memory
- One simple solution is to divide the page table into smaller units
  - Hierarchical Paging
  - Hashed Page Tables
  - Inverted Page Tables
- Will be discussed in more advanced courses

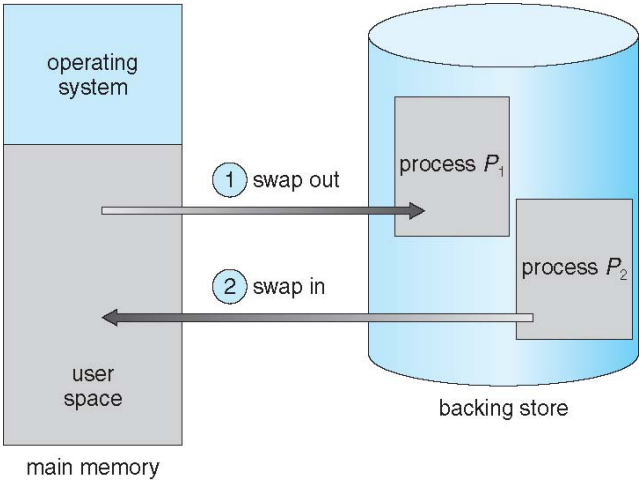
Swapping

- A process can be swapped temporarily out of memory to a backing store, and then brought back into memory for continued execution
  - Total physical memory space of processes can exceed physical memory
- Backing store: fast disk large enough to accommodate copies of all memory images for all users; must provide direct access to these memory images
- Roll out, roll in: swapping variant used for priority-based scheduling algorithms; lower-priority process is swapped out so higher-priority process can be loaded and executed
- Major part of swap time is transfer time; total transfer time is directly proportional to the amount of memory swapped
- System maintains a ready queue of ready-to-run processes which have memory images on disk

Swapping

- Does the swapped out process need to swap back in to same physical addresses?
- Depends on address binding method
  - Plus consider pending I/O to / from process memory space
- Modified versions of swapping are found on many systems (i.e., UNIX, Linux, and Windows)
  - Swapping normally disabled
  - Started if more than threshold amount of memory allocated
  - Disabled again once memory demand reduced below threshold

Schematic View of Swapping



Context Switch Time including Swapping

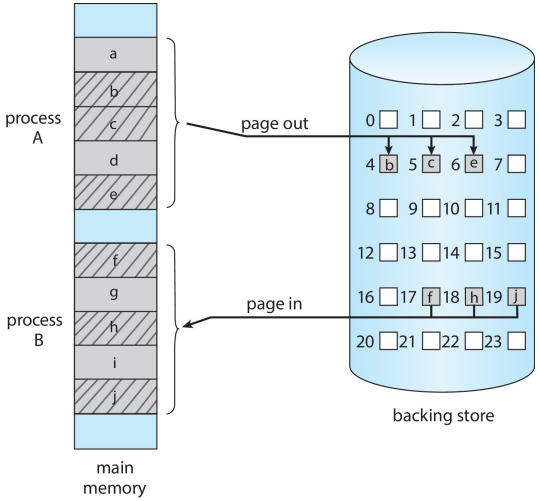
UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- If next processes to be put on CPU is not in memory, need to swap out a process and swap in target process
- Context switch time can then be very high
- 100MB process swapping to hard disk with transfer rate of 50MB/sec
  - Swap out time of 2000 ms
  - Plus swap in of same sized process
  - Total context switch swapping component time of 4000ms (4 seconds)
- Can reduce if reduce size of memory swapped – by knowing how much memory really being used
  - System calls to inform OS of memory use via `request_memory()` and `release_memory()`

Context Switch Time and Swapping

- Other constraints as well on swapping
  - Pending I/O – can't swap out as I/O would occur to wrong process
  - Or always transfer I/O to kernel space, then to I/O device
  - Known as double buffering, adds overhead
- Standard swapping not used in modern operating systems
  - But modified version common
  - Swap only when free memory extremely low

Swapping with Paging



UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

Virtual Memory

Background

- Code needs to be in memory to execute, but entire program rarely used
  - Error code, unusual routines, large data structures
- Entire program code not needed at same time
- Consider ability to execute partially-loaded program
  - Program no longer constrained by limits of physical memory
  - Each program takes less memory while running → more programs run at the same time
  - Increased CPU utilization and throughput with no increase in response time or turnaround time
  - Less I/O needed to load or swap programs into memory → each user program runs faster

Virtual memory

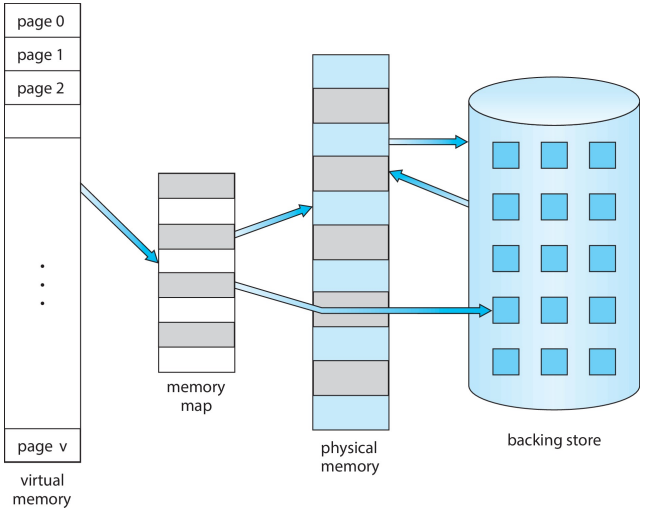
- Virtual memory: separation of user logical memory from physical memory
- Only part of the program needs to be in memory for execution
- Logical address space can therefore be much larger than physical address space
- Allows address spaces to be shared by several processes
- Allows for more efficient process creation
- More programs running concurrently
- Less I/O needed to load or swap processes

Virtual memory

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

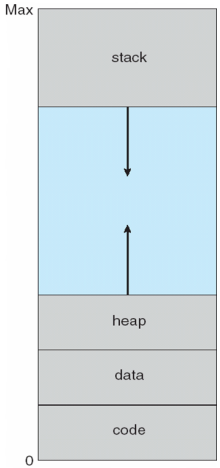
- Virtual address space: logical view of how process is stored in memory
  - Usually start at address 0, contiguous addresses until end of space
  - Meanwhile, physical memory organized in page frames
  - MMU must map logical to physical
- Virtual memory can be implemented via
  - Demand paging
  - Demand segmentation

Virtual Memory > Physical Memory

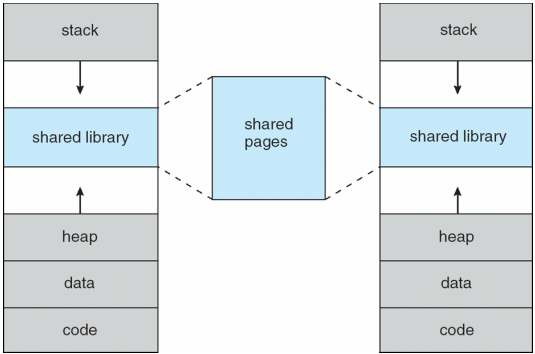


Virtual-address Space

- Usually logical address space for stack to start at Max logical address and grow “down” while heap grows “up”
  - Maximizes address space use
  - Unused address space between the two is hole
  - No physical memory needed until heap or stack grows to a given new page
- Enables sparse address spaces with holes left for growth, dynamically linked libraries, etc.
- System libraries shared via mapping into virt. addr. space
- Shared memory by mapping pages read-write into virt. addr. space
- Pages can be shared during `fork()`
  - Faster process creation



Shared Library Using Virtual Memory



Inspecting memory usage

The `free` command displays memory usage `free -h`

```
       total  used  free  shared  buff/cache  available
Mem:   15G    9,7G  1,3G  2,2G    6,8G        5,6G
Swap:  975M   24K   975M
```

- total: Total amount of physical memory (RAM) installed.
- used: Memory currently in use by running processes.
- free: Memory that is completely unused.
- shared: Memory used by tmpfs and shared memory.
- buff/cache: Memory used for buffers and disk cache.
- available: Estimate of how much memory is available for starting new applications, without swapping.

Looking at process memory

```
cat /proc/$$/statm
2376 1670 1035 218 0 656 0
```

Columns of the file

- Size: total program size (in pages)
- Resident: resident set size (in pages)
- Shared: number of resident shared pages
- Text: text (code) size (in pages)
- Lib: library size (in pages), usually 0
- Data: data + stack size (in pages)
- Dirty: dirty pages (in pages), usually 0

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

Demand Paging

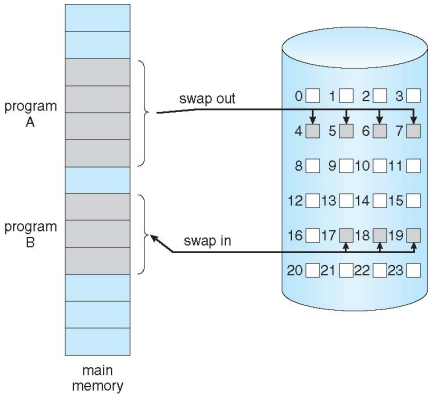
Demand Paging

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Could bring entire process into memory at load time
- Or bring a page into memory only when it is needed
  - Less I/O needed, no unnecessary I/O
  - Less memory needed
  - Faster response
  - More users
- Similar to paging system with swapping (diagram on right)
- Page is needed → reference to it
  - Invalid reference → abort
  - Not-in-memory → bring to memory
- Lazy swapper: never swaps a page into memory unless page will be needed
  - Swapper that deals with pages is a pager

Demand Paging

- Could bring entire process into memory at load time
- Or bring a page into memory only when it is needed
  - Less I/O needed, no unnecessary I/O
  - Less memory needed
  - Faster response
  - More users
- Similar to paging system with swapping

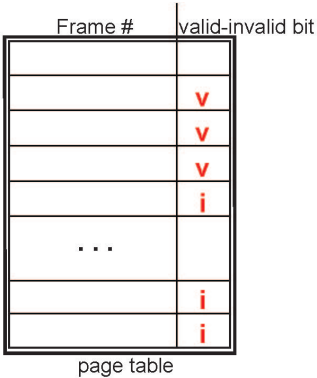


Basic Concepts

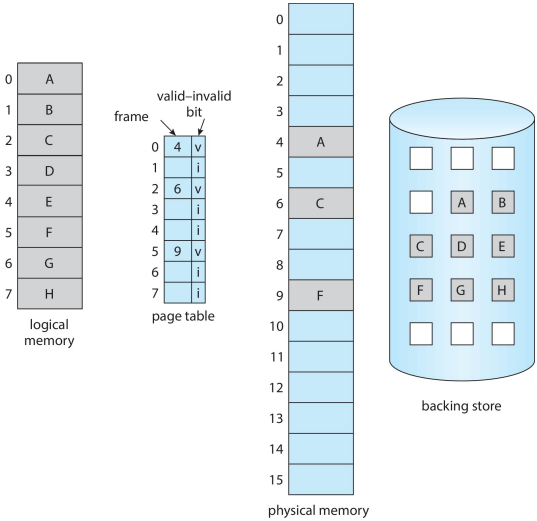
- With swapping, pager guesses which pages will be used before swapping out again
- Instead, pager brings in only those pages into memory
- How to determine that set of pages?
  - Need new MMU functionality to implement demand paging
- If pages needed are already memory resident
  - No difference from non demand-paging
- If page needed and not memory resident
  - Need to detect and load the page into memory from storage
  - Without changing program behavior
  - Without programmer needing to change code

Valid/Invalid Bit

- With each page table entry a valid/invalid bit is associated
  - *v* → in-memory (memory resident)
  - *i* → not-in-memory
- Initially valid/invalid bit is set to *i* on all entries
- Example of a page table snapshot:
- During MMU address translation
  - if valid/invalid bit in page table entry is i → page fault



Some Pages Not in Memory

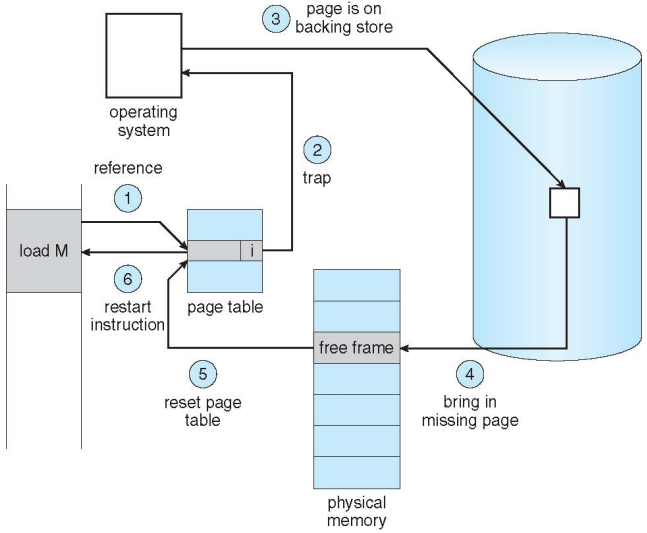


Steps in Handling Page Fault

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- If there is a reference to a page, first reference to that page will trap to operating system
  - Page fault
- Operating system looks at another table to decide
  - Invalid reference → abort
  - Just not in memory
- Find free frame
- Swap page into frame via scheduled disk operation
- Reset tables to indicate page now in memory
  - Set validation bit = *v*
- Restart the instruction that caused the page fault

Steps in Handling a Page Fault



Aspects of Demand Paging

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Extreme case: start process with no pages in memory
  - OS sets instruction pointer to first instruction of process, non-memory-resident → page fault
  - Repeat for every other process pages on first access
  - Pure demand paging
- Actually, a given instruction could access multiple pages → multiple page faults
  - Consider fetch and decode of instruction which adds 2 numbers from memory and stores result back to memory
  - Pain decreased because of locality of reference
- Hardware support needed for demand paging
  - Page table with valid / invalid bit
  - Secondary memory (swap device with swap space)
  - Instruction restart

Free-Frame List

- When a page fault occurs, the operating system must bring the desired page from secondary storage into main memory
- Most operating systems maintain a free-frame list
  - Pool of free frames for satisfying such requests

- Operating system typically allocate free frames using a technique known as zero-fill-on-demand
  - The content of the frames zeroed-out before being allocated
- When a system starts up, all available memory is placed on the free-frame list.

Stages in Demand Paging: Worse Case

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

1. Trap to the operating system
2. Save the user registers and process state
3. Determine that the interrupt was a page fault
4. Check that the page reference was legal and determine the location of the page on the disk
5. Issue a read from the disk to a free frame
   - Wait in a queue for this device until the read request is serviced
   - Wait for the device seek and/or latency time
   - Begin the transfer of the page to a free frame

Stages in Demand Paging

6. While waiting, allocate the CPU to some other user
7. Receive an interrupt from the disk I/O subsystem (I/O completed)
8. Save the registers and process state for the other user
9. Determine that the interrupt was from the disk
10. Correct the page table and other tables to show page is now in memory
11. Wait for the CPU to be allocated to this process again
12. Restore the user registers, process state, and new page table, and then resume the interrupted instruction

Performance of Demand Paging

- Three major activities
  - Service the interrupt – careful coding means just several hundred instructions needed
  - Read the page – lots of time
  - Restart the process – again just a small amount of time
- Page Fault Rate $p \in [0, 1]$
  - if $p = 0$ no page faults
  - if $p = 1$, every reference is a fault
- Effective Access Time (EAT)

$$\begin{aligned} EAT = & (1 - p) \times T_{mem-access} + \\ & + p \times (T_{page-fault-overhead} + \\ & + T_{swap-page-out} \\ & + T_{swap-page-in}) \end{aligned}$$

Demand Paging Example

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Memory access time = 200 nanoseconds
- Average page-fault service time = 8 milliseconds
- $EAT = (1 - p) \times 200 + p \times 8ms$
  $= (1 - p) \times 200 + p \times 8 \cdot 10^6$
  $= 200 + p \times 7999800$
- If one access out of 1,000 causes a page fault, then
  - EAT = 8.2 microseconds
  - This is a slowdown by a factor of 40!!
- If want performance degradation $< 10\%$
  - $220 > 200 + 7,999,800 \times p \rightarrow 20 > 7999800 \times p$
  - $p < 0.0000025$
  - less than one page fault in every 400×10^3 memory accesses

Demand Paging Optimizations

- Swap space I/O faster than file system I/O even if on the same device
  - Swap allocated in larger chunks, less management needed than file system
- Copy entire process image to swap space at process load time
  - Then page in and out of swap space
  - Used in older BSD Unix
- Demand page in from program binary on disk, but discard rather than paging out when freeing frame
  - Used in Solaris and current BSD
  - Still need to write to swap space
    - Pages not associated with a file (like stack and heap): anonymous memory
    - Pages modified in memory but not yet written back to the file system
- Mobile systems
  - Typically don't support swapping
  - Instead, demand page from file system and reclaim read-only pages (e.g., code)

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

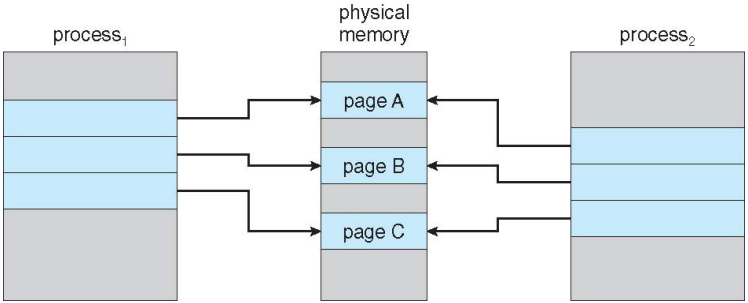
Copy-on-Write

Copy-on-Write

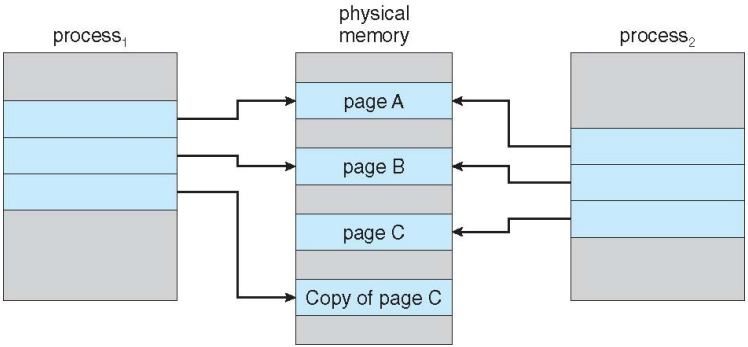
UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Copy-on-Write (COW) allows both parent and child processes to initially share the same pages in memory
  - If either process modifies a shared page, only then is the page copied
- COW allows more efficient process creation as only modified pages are copied
- In general, free pages are allocated from a pool of zero-fill-on-demand pages
  - Pool should always have free frames for fast demand page execution
  - Don't want to have to free a frame as well as other processing on page fault
  - Why zero-out a page before allocating it?
- `vfork()` variation on `fork()` system call has parent suspend and child using copy-on-write address space of parent
  - Designed to have child call exec()
  - Very efficient

Before Process 1 Modifies Page C



After Process 1 Modifies Page C



If no Free Frame Are Available

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Used up by process pages
- Also in demand from the kernel, I/O buffers, etc
- How much to allocate to each?
- Page replacement: find some page in memory, but not really in use, page it out
  - Algorithm: terminate? swap out? replace the page?
  - Performance: want an algorithm which will result in minimum number of page faults
- Same page may be brought into memory several times
- Will be discussed in more advanced courses